Commitment 시스템 상세 분석

개요

Proof of SQL의 commitment 시스템은 영지식 증명의 핵심 구성 요소로, 데이터베이스의 컬럼과 테이블에 대한 암호학적 commitment를 생성하고 관리합니다. 이 시스템은 데이터의 무결성을 보장하면서도 실제 데이터를 노출하지 않는 방식으로 증명을 가능하게 합니다.

파일 구조 분석

`crates/proof-of-sql/src/base/commitment/mod.rs`

이 파일은 commitment 시스템의 메인 모듈로, 다양한 commitment 관련 타입과 트레이트를 정의합니다.

주요 임포트 및 의존성

```
//! Types for creation and utilization of cryptographic commitments to proof-of-

use  crate::base::scalar::Scalar;

use alloc::vec::Vec;
```

라인별 분석:

- `//!` : 모듈 레벨 문서 주석으로, 이 모듈이 proof-of-sql 데이터에 대한 암호학적 commitment 생성과 활용을 위한 타입들을 제공함을 설명
- `use crate::base::scalar::Scalar` : 스칼라 연산을 위한 기본 트레이트 임포트
- `use alloc::vec::Vec` : no_std 환경에서도 동작하도록 alloc 크레이트의 Vec 사용

Blitzar 통합

```
#[cfg(feature = "blitzar")]
pub  use blitzar::{
compute::{init_backend, init_backend_with_config, BackendConfig},
proof::InnerProductProof,
};
```

라인별 분석:

- `#[cfg(feature = "blitzar")]` : blitzar 기능이 활성화된 경우에만 컴파일
- `blitzar::compute` : GPU 가속 연산을 위한 백엔드 초기화 함수들
- `blitzar::proof::InnerProductProof` : 내적 증명을 위한 구조체
- **Blitzar**: GPU 가속 암호학적 연산을 위한 라이브러리로, 대규모 commitment 연산을 효율적으로 처리

모듈 구조

```
mod committable_column;
pub use committable_column::CommittableColumn;

mod vec_commitment_ext;
pub use vec_commitment_ext::{NumColumnsMismatch, VecCommitmentExt};
```

모듈별 역할:

- `committable_column` : 다양한 데이터 타입을 commitment 가능한 형태로 변환

- `vec_commitment_ext` : 벡터 형태의 commitment 확장 기능
- `column_bounds` : 컬럼의 범위와 경계 관리
- `column_commitment_metadata` : 컬럼 commitment의 메타데이터 관리
- `table_commitment` : 전체 테이블에 대한 commitment

핵심 트레이트: `Commitment`

```
pub trait Commitment:
AddAssign
+ SubAssign
+ Sized
+ Default
+ Clone
+ core::ops::Neg<Output = Self>
+ Eq
+ core::ops::Sub<Output = Self>
+ core::fmt::Debug
+ core::marker::Sync
+ core::marker::Send
{
```

트레이트 분석:

- `AddAssign` , `SubAssign` : commitment 간의 덧셈/뺄셈 연산 지원
- `Neg` : commitment의 음수 연산 (영지식 증명에서 중요한 연산)

- `Sync`, `Send`: 멀티스레드 환경에서 안전한 사용을 위한 마커 트레이트
- 이러한 연산들은 commitment의 동형성(homomorphic) 특성을 활용

연관 타입 정의

```
/// The associated scalar that the commitment is for.
/// There are multiple possible commitment schemes for a scalar, but only one sc
type Scalar: Scalar
+ for<'a> core::ops::Mul<&'a Self, Output = Self>
+ core::ops::Mul<Self, Output = Self>
+ serde::Serialize
+ for<'a> serde::Deserialize<'a>;

/// The public setup for the commitment scheme.
type PublicSetup<'a>;
```

연관 타입 분석:

- `Scalar`: commitment가 작동하는 스칼라 필드
- `PublicSetup`: commitment 스킴의 공개 설정 매개변수 (예: 생성원, 공개키 등)
- 스칼라와 commitment 간의 곱셈 연산 지원 (스칼라 곱셈)

핵심 메서드: `compute_commitments`

```
/// Compute the commitments for the given columns.
///
```

```
/// The resulting commitments are written to the slice in `commitments`, which i
/// `commitments` is expected to have the same length as `committable_columns` a
/// The length of each [`CommittableColumn`] should be the same.
///
/// `offset` is the amount that `committable_columns` is "offset" by. Logically
fn compute_commitments(
committable_columns: &[CommittableColumn],
offset: usize,
setup: &Self::PublicSetup<'_>,
) -> Vec<Self>;
```

메서드 분석:

- **목적**: 주어진 컬럼들에 대한 commitment 계산
- **매개변수**:
- `committable_columns` : commitment할 컬럼 데이터 배열
- `offset` : 논리적 오프셋 (앞쪽에 0을 추가하는 효과)
- `setup` : 공개 설정 매개변수
- **반환값**: 각 컬럼에 대응하는 commitment 벡터
- **중요성**: 이 메서드가 실제 암호학적 commitment 생성의 핵심

직렬화 메서드

```
/// Converts the commitment to bytes that will be appended to the transcript.
```

```
///
/// This is also useful for serialization purposes.
fn to_transcript_bytes(&self) -> Vec<u8>;
```

메서드 분석:

- **목적**: commitment를 바이트 배열로 변환
- **용도**:
- Fiat-Shamir 변환을 위한 transcript에 추가
- 네트워크 전송이나 저장을 위한 직렬화
- **보안성**: transcript에 포함되어 무작위성 생성에 기여

CommittableColumn 분석

데이터 타입 변환

`CommittableColumn` 은 다양한 SQL 데이터 타입을 commitment 가능한 형태로 변환하는 열거형입니다:

```
#[non_exhaustive]
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum CommittableColumn<'a> {
/// Borrowed Bool column, mapped to `bool`.
Boolean(&'a [bool]),
/// Borrowed `Byte` column, mapped to `u8`.
Uint8(&'a [u8]),
/// Borrowed `TinyInt` column, mapped to `i8`.
```

```
TinyInt(&'a [i8]),
// ... 기타 정수 타입들
/// Column of big ints for committing to, montgomery-reduced from a Scalar colum
Scalar(Vec<[u64; 4]>),
/// Column of limbs for committing to scalars, hashed from a `VarChar` column.
VarChar(Vec<[u64; 4]>),
}
```

핵심 특징:

- **차용(Borrowing) vs 소유(Owned)**: 간단한 타입은 차용, 복잡한 타입은 소유
- **Montgomery 형태**: 스칼라 값들은 Montgomery 형태로 변환되어 효율적인 연산 지원
- **해싱**: 문자열과 바이너리 데이터는 해시를 통해 스칼라로 변환
- **4-limb 표현**: 큰 수들은 64비트 limb 4개로 표현 (256비트)

Blitzar 통합

```
#[cfg(feature = "blitzar")]
impl<'a, 'b> From<&'a CommittableColumn<'b>> for Sequence<'a> {
fn from(value: &'a CommittableColumn<'b>) -> Self {
match value {
CommittableColumn::Uint8(ints) => Sequence::from(*ints),
CommittableColumn::TinyInt(ints) => Sequence::from(*ints),
// ... 각 타입별 변환
CommittableColumn::Scalar(limbs) => Sequence::from(limbs),
```

```
}

}

}
```

GPU 가속 통합:

- `Sequence` : Blitzar의 GPU 연산을 위한 데이터 구조
- 각 데이터 타입을 GPU에서 처리 가능한 형태로 변환
- 대규모 commitment 연산의 병렬 처리 지원

CommitmentEvaluationProof 트레이트

증명 생성과 검증

```
pub trait CommitmentEvaluationProof {

type Scalar: Scalar + Serialize + for<'a> Deserialize<'a>;

type Commitment: for<'a> Commitment<Scalar = Self::Scalar, PublicSetup<'a> = Sel

type Error;

type ProverPublicSetup<'a>: Copy;

type VerifierPublicSetup<'a>: Copy;
```

트레이트 설계:

- **제네릭 설계**: 다양한 commitment 스킴에 대해 일관된 인터페이스 제공
- **증명자/검증자 분리**: 서로 다른 설정 매개변수 사용 가능
- **직렬화 지원**: 네트워크를 통한 증명 전송 지원

증명 생성 메서드

```
/// Create a new proof.
///
/// Note: `b_point` must have length `nu`, where `2^nu` is at least the length o
/// `b_point` are the values for the variables that are being evaluated.
/// The resulting evaluation is the inner product of `a` and `b`, where `b` is t
fn new(
transcript: &mut impl Transcript,
a: &[Self::Scalar],
b_point: &[Self::Scalar],
generators_offset: u64,
setup: &Self::ProverPublicSetup<'_>,
) -> Self;
```

수학적 배경:

- **내적 증명**: `a` 와 `b` 의 내적을 증명
- **벡터 확장**: `b_point` 는 `b` 의 압축된 형태
- **크기 제약**: `2^nu ≥ len(a)` 조건 (이진 트리 구조)
- **생성원 오프셋**: 서로 다른 증명 간의 독립성 보장

배치 검증

```
/// Verify a batch proof. This can be more efficient than verifying individual p
```

```rust
fn verify_batched_proof(
&self,
transcript: &mut impl Transcript,
commit_batch: &[Self::Commitment],
batching_factors: &[Self::Scalar],
evaluations: &[Self::Scalar],
b_point: &[Self::Scalar],
generators_offset: u64,
table_length: usize,
setup: &Self::VerifierPublicSetup<'_>,
) -> Result<(), Self::Error>;
```

배치 최적화:

- **효율성**: 여러 증명을 한 번에 검증하여 성능 향상
- **배치 인수**: 각 commitment에 대한 가중치
- **선형 결합**: 여러 commitment를 선형 결합하여 단일 검증으로 축약

암호학적 원리

Commitment 스킴의 특성

1. **Binding**: 한 번 commitment를 생성하면 다른 값으로 변경 불가
2. **Hiding**: commitment만으로는 원본 값을 알 수 없음
3. **Homomorphic**: commitment 간의 연산이 원본 값의 연산과 대응

영지식 증명에서의 역할

1. **데이터 은닉**: 실제 데이터베이스 내용을 노출하지 않음
2. **무결성 보장**: 데이터가 변조되지 않았음을 증명
3. **효율적 검증**: 전체 데이터를 다시 계산하지 않고도 검증 가능

성능 최적화

GPU 가속 (Blitzar)

- **병렬 처리**: 수천 개의 commitment를 동시에 계산
- **메모리 효율성**: GPU 메모리를 효율적으로 활용
- **배치 처리**: 여러 컬럼을 한 번에 처리

메모리 관리

- **Zero-copy**: 가능한 경우 데이터 복사 없이 참조 사용
- **지연 할당**: 필요한 경우에만 메모리 할당
- **타입별 최적화**: 각 데이터 타입에 맞는 최적화된 처리

보안 고려사항

암호학적 가정

- **이산 로그 문제**: 타원 곡선에서의 이산 로그 문제의 어려움에 기반
- **페어링 친화적 곡선**: BN254 등의 페어링 친화적 곡선 사용
- **신뢰할 수 있는 설정**: 일부 스킴에서는 신뢰할 수 있는 설정 필요

구현 보안

- **상수 시간 연산**: 타이밍 공격 방지
- **메모리 보안**: 민감한 데이터의 안전한 소거
- **난수 생성**: 암호학적으로 안전한 난수 사용

이 commitment 시스템은 Proof of SQL의 핵심 보안 기반을 제공하며, 효율적이고 안전한 영지식 증명을 가능하게 합니다.